

Documento de Arquitetura

Modernizacao de Sistema Legado - Cooperativa Financeira Alfa

Projeto Final COBOL | Acelera Maker / Montreal | Julho 2026

#	Secao
1	Visao Geral
2	Arquitetura da Solucao
3	Componentes
4	Protocolo de Comunicacao
5	Fluxo de Execucao
6	Decisoes Arquiteturais
7	Requisitos Atendidos
8	Estrutura do Projeto
9	Stack de Tecnologias

1. Visao Geral

A Cooperativa Financeira Alfa necessita modernizar seu sistema legado de cadastro de clientes sem substitui-lo, mantendo compatibilidade com o processamento existente. A solucao adotada expoe o programa COBOL como um servico consumivel por novas aplicacoes, por meio de uma API REST desenvolvida em ASP.NET Core 8.

O objetivo central e permitir que atendentes consultem e atualizem informacoes de clientes de maneira simples, enquanto o componente legado (COBOL) continua sendo responsavel pelo processamento das regras de negocio cadastrais.

2. Arquitetura da Solucao

A arquitetura e composta por tres camadas que se comunicam de forma sequencial:

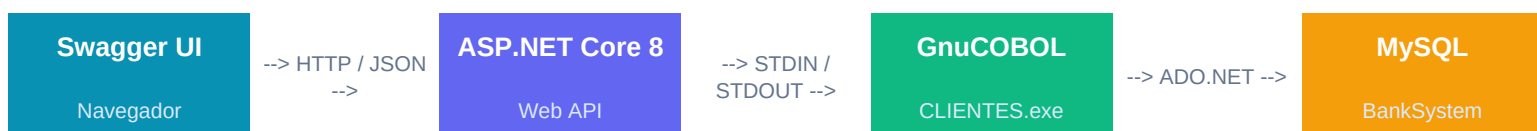


Figura 1 - Diagrama de arquitetura: Swagger UI -> ASP.NET Core 8 -> GnuCOBOL -> MySQL

Swagger UI: Interface de consumo da API gerada automaticamente pelo Swashbuckle. Em producao, qualquer cliente HTTP pode consumir os endpoints REST.

ASP.NET Core 8 (Web API): Orquestra o fluxo completo: recebe a requisicao HTTP, busca dados no MySQL, repassa ao COBOL para processamento e persiste as alteracoes apos confirmacao do programa legado.

GnuCOBOL (CLIENTES.exe): Recebe dados via STDIN, executa a logica de negocio e retorna o resultado via STDOUT. Nao acessa o banco diretamente.

MySQL (BankSystem): Banco de dados acessado exclusivamente pelo .NET, responsavel por todas as operacoes de leitura e escrita.

3. Componentes

3.1 ClienteRepository.cs

Servico responsavel pelo acesso ao MySQL. Usa MySql.Data (v9.7.0) com queries parametrizadas para prevenir SQL Injection.

Metodo	Descricao
BuscarPorCodigoAsync(codigo)	Retorna um Cliente ou null se nao encontrado
AtualizarContatoAsync(codigo, tel, email)	Atualiza telefone e e-mail, retorna bool

3.2 CobolService.cs

Servico responsavel pela comunicacao com o CLIENTES.exe via System.Diagnostics.Process com redirecionamento de STDIN/STDOUT.

Metodo	Descricao
ExecutarAsync(input)	Envia string ao COBOL e retorna resposta bruta
ConsultarAsync(cliente)	Monta input CONSULTAR ... e parseia OK ... ou NOTFOUND
AtualizarAsync(cliente, tel, email)	Monta input ATUALIZAR ... e parseia ATUALIZADO ...

3.3 ClientesController.cs

Expoe dois endpoints REST documentados via OpenAPI 3.0:

Metodo	Rota	Descricao	Respostas HTTP
GET	/api/Clientes/{codigo}	Consulta cliente pelo codigo	200, 404, 500
PUT	/api/Clientes/{codigo}	Atualiza telefone e e-mail	200, 404, 500

3.4 CLIENTES.cbl

Programa COBOL compilado com GnuCOBOL 2.0 (32 bits) via OpenCobolIDE 4.7.6. Le uma linha do STDIN, processa a acao solicitada e escreve o resultado no STDOUT.

4. Protocolo de Comunicacao .NET x COBOL

A comunicacao entre .NET e COBOL e feita por strings delimitadas por pipe (|) via STDIN/STDOUT. O formato foi escolhido por ser simples, sem dependencias externas e compativel com qualquer versao do GnuCOBOL.

Formato de entrada (STDIN)

ACAO|CODIGO|NOME|TELEFONE|EMAIL|NOVO_TELEFONE|NOVO_EMAIL

Campo	Descricao	Exemplo
ACAO	CONSULTAR ou ATUALIZAR	CONSULTAR
CODIGO	Codigo do cliente (5 digitos)	00001
NOME	Nome completo	Joao Silva
TELEFONE	Telefone atual	11999990001
EMAIL	E-mail atual	joao@email.com
NOVO_TELEFONE	Novo telefone (ATUALIZAR)	21888880001
NOVO_EMAIL	Novo e-mail (ATUALIZAR)	novo@email.com

Formato de saida (STDOUT)

Resposta	Situacao
OK CODIGO NOME TELEFONE EMAIL	Consulta bem-sucedida
ATUALIZADO CODIGO NOVO_TELEFONE NOVO_EMAIL	Atualizacao confirmada
NOTFOUND	Cliente nao encontrado
ERRO MOTIVO	Erro no processamento

5. Fluxo de Execucao

5.1 GET /api/Clientes/{codigo}

#	Responsavel	Acao
1	.NET Controller	Recebe requisicao HTTP GET

2	ClienteRepository	Busca cliente no MySQL por codigo
3	.NET Controller	Retorna 404 se cliente nao encontrado
4	CobolService	Monta string CONSULTAR ... e inicia CLIENTES.exe
5	CLIENTES.exe	Processa via STDIN, retorna OK ... via STDOUT
6	CobolService	Parseia resposta do COBOL
7	.NET Controller	Retorna 200 com JSON do cliente ou 500 em caso de erro

5.2 PUT /api/Clientes/{codigo}

#	Responsavel	Acao
1	.NET Controller	Recebe requisicao HTTP PUT com body JSON
2	ClienteRepository	Busca cliente atual no MySQL
3	.NET Controller	Retorna 404 se cliente nao encontrado
4	CobolService	Monta string ATUALIZAR ... e inicia CLIENTES.exe
5	CLIENTES.exe	Valida e processa, retorna ATUALIZADO ...
6	ClienteRepository	Persiste novos dados no MySQL apos confirmacao COBOL
7	.NET Controller	Retorna 200 com cliente atualizado

6. Decisoes Arquiteturais

.NET acessa o MySQL, nao o COBOL: O GnuCOBOL 2.0 nao suporta Embedded SQL de forma estavel no ambiente utilizado. Centralizar o acesso ao banco no .NET simplifica o gerenciamento de conexoes, permite queries parametrizadas e mantem o COBOL responsavel apenas pela logica de negocio.

Comunicacao via STDIN/STDOUT: Alternativas como sockets TCP ou arquivos temporarios foram descartadas por exigirem configuracao adicional. O pipe de texto e nativo, sem instalacao extra, e funciona em qualquer ambiente onde o .exe possa ser executado.

Formato de dados: string delimitada por pipe: JSON foi considerado mas exigiria um parser no COBOL. O formato pipe e trivial de ler e escrever em COBOL com UNSTRING/STRING, sem bibliotecas externas.

COBOL como validador, .NET como persistidor: O COBOL confirma se a operacao e valida (retorna ATUALIZADO ou ERRO) antes de o .NET gravar no banco. Isso preserva a responsabilidade do sistema legado sobre as regras cadastrais.

Swagger na raiz (RoutePrefix vazio): Facilita o acesso durante desenvolvimento e demonstracao sem necessidade de navegar para /swagger/index.html manualmente.

7. Requisitos Atendidos

Requisito	Como e atendido
Consultar cliente pelo codigo	GET /api/Clientes/{codigo} - MySQL + COBOL
Exibir codigo, nome, telefone, e-mail	Response JSON com os 4 campos obrigatorios
Alterar telefone e e-mail	PUT - COBOL valida, .NET persiste no MySQL
Informar cliente nao encontrado	HTTP 404 com mensagem descritiva
Persistir alteracoes	ClienteRepository.AtualizarContatoAsync
Utilizar .NET e COBOL	ASP.NET Core 8 + GnuCOBOL 2.0
Interface para o usuario	Swagger UI gerado automaticamente
Possibilitar integracao futura	API REST padrao, documentada via OpenAPI 3.0
Estrutura organizada	Models / Services / Controllers separados
Definicao unica da estrutura	Classe Cliente.cs compartilhada por toda a solucao

8. Estrutura do Projeto

```
legacy-client-modernization/  
- api/ClientesApi/  
- Controllers/ClientesController.cs - Endpoints GET e PUT  
- Models/Cliente.cs - Model + Request DTO  
- Services/ClienteRepository.cs - Acesso MySQL  
- Services/CobolService.cs - Integracao COBOL  
- Program.cs - DI + Swagger  
- appsettings.json - Connection string + caminho exe  
- cobol/  
- CLIENTES.cbl - Programa COBOL  
- bin/CLIENTES.exe - Compilado GnuCOBOL 2.0  
- testes/ClientesApi.Testes/  
- ClientesControllerTests.cs - 8 testes xUnit  
- ClientesApi.Tests.csproj  
- docs/ - Documentacao
```

9. Stack de Tecnologias

Tecnologia	Versao	Papel
ASP.NET Core	8.0	Framework Web API
GnuCOBOL	2.0	Processamento legado
MySQL	8.0	Banco de dados
MySql.Data	9.7.0	Driver ADO.NET para MySQL

Swashbuckle	6.9.0	Geracao do Swagger / OpenAPI 3.0
xUnit	2.9.0	Framework de testes automatizados
Moq	4.20	Mocks nos testes de integracao
OpenCobolIDE	4.7.6	IDE de compilacao COBOL